Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский университет ИТМО



Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа № 1
«Интерфейсы ввода-вывода общего назначения (GPIO)»

по дисциплине

«Проектирование вычислительных систем»

Вариант № 5: «Кодовый замок»

Выполнили: Ястребов-Амирханов Алекси, Исупов Никита Александрович
Группа: P3432/P3431
Преподаватель: Ключев Владислав Аркадьевич

Санкт-Петербург,

2026

Оглавление

1. Цели работы **3**
2. Описание варианта задания **3**
2.1. Часть 1. Разработка прикладной программы «Кодовый замок» 3
2.2. Часть 2. Изучение логики работы портов GPIO 4
3. Аппаратная платформа и схема подключения стенда SDK-1.1M **4**
4. Архитектурное описание программы и алгоритмов **5**
4.1. Диспетчеризация и принципы кооперативной многозадачности 6
4.2. Алгоритм фильтрации дребезга и классификации нажатий кнопки 6
4.3. Организация светодиодной индикации 8
4.4. Логика конечного автомата кодового замка 8
5. Анализ регистров GPIO STM32F427 и реализация драйвера my_gpio **10**
5.1. Архитектура и назначение регистров GPIO 10
5.2. Реализация безопасной инициализации без выбросов (Glitch-Free Reconfiguration). 11
5.3. Преимущества использования регистра BSRR над ODR 12
6. Методика и результаты экспериментальной верификации **13**
7. Выводы **15**

1. Цели работы

1. Получить базовые практические знания об архитектуре, схемотехнике и принципах низкоуровневого программирования аппаратных средств учебного стенда-конструктора SDK-1.1M на базе микроконтроллера `STM32F427VIT6` (процессорное ядро ARM Cortex-M4).

2. Изучить схемотехническую организацию и принципы функционирования портов ввода-вывода общего назначения (GPIO) микроконтроллеров семейства STM32F4, а также структуру, назначение и способы конфигурации управляющих регистров специального назначения (`MODER`, `OTYPER`, `OSPEEDR`, `PUPDR`, `IDR`, `ODR`, `BSRR`).

3. Разработать прикладное встроенное программное обеспечение для решения задачи «Кодовый замок» (Вариант 5) с использованием архитектурной модели кооперативной многозадачности без блокирующих задержек, без использования аппаратных прерываний и дополнительных аппаратных таймеров (только системное квантование времени `SysTick` / `HAL_GetTick()`).

4. Освоить схемотехнические и программные методы подавления дребезга механических контактов кнопок на основе конечных автоматов (FSM) и алгоритмы классификации коротких («точка») и длинных («тире») нажатий.

5. Разработать и внедрить независимый собственный регистровый драйвер GPIO (без использования стандартных библиотек HAL и LL), поддерживающий безопасную повторную конфигурацию выводов с гарантированным исключением переходных импульсных выбросов (glitch-free).

2. Описание варианта задания

2.1. Часть 1. Разработка прикладной программы «Кодовый замок»

В соответствии с заданием варианта № 5 требуется разработать программное обеспечение для стенда SDK-1.1M, реализующее алгоритм функционирования системы «Кодовый замок» со следующими характеристиками:

- **Секретная последовательность:** состоит не менее чем из 8 коротких и длинных нажатий кнопки боковой панели (линия `PC15`). В настоящей работе задана кодовая комбинация из 8 нажатий: [К, К, Д, К, Д, Д, К, К], где «К» обозначает короткое нажатие (длительность импульса от 50 до 449 мс), а «Д» — длинное нажатие (длительность импульса не менее 450 мс).
- **Индикация корректного ввода:** каждый правильно введенный очередной элемент последовательности сопровождается одиночной вспышкой желтого светодиода длительностью 200 мс.
- **Открытие замка:** при безошибочном вводе всей последовательности из 8 элементов включается зеленый светодиод на 3 секунды, сигнализируя об успешном открытии замка. На протяжении этого интервала любые нажатия на кнопку игнорируются. По истечении 3 секунд светодиод выключается, и замок возвращается в исходный режим ожидания ввода.
- **Реакция на ошибку ввода:** при нажатии, не совпадающем с ожидаемым элементом кодовой последовательности, выдается одиночная вспышка красного светодиода длительностью 250 мс, текущая позиция ввода немедленно сбрасывается в начало (индекс равен 0), а счетчик неудачных попыток увеличивается на единицу.
- **Режим тревоги и блокировки:** после трех неправильных попыток ввода подряд активируется аварийный режим. Красный светодиод начинает непрерывно мигать с

периодом 200 мс на протяжении не менее 5 секунд (выбрано 5.5 секунд). Во время тревоги любые нажатия кнопки полностью игнорируются. После завершения тревоги счетчик ошибок обнуляется, и замок переходит в состояние ожидания ввода.

- **Таймаут неполного ввода:** если ввод кодовой комбинации был начат, но следующее нажатие не последовало в течение 5 секунд, желтый светодиод выдает длинный световой импульс длительностью 1500 мс, после чего система сбрасывает введенные элементы в исходное состояние.
- **Неблокирующее исполнение:** в программе и драйверах запрещено использование активных ожиданий (функции `HAL_Delay()`). Опрос кнопки и светодиодные эффекты должны выполняться псевдопараллельно в кооперативном стиле.

2.2. Часть 2. Изучение логики работы портов GPIO

Разработать собственную реализацию драйвера GPIO без вызова функций и макросов библиотек HAL и LL (за исключением базовой структуры адресов регистров `GPIO_TypeDef`). Драйвер должен обеспечивать:

- Инициализацию режимов работы портов (вход, выход Push-Pull или Open-Drain, аналоговый режим, конфигурация подтяжек Pull-Up/Pull-Down, выбор скорости);
- Считывание текущих параметров конфигурации конкретного вывода;
- Атомарную побитовую запись через регистр `BSRR`;
- Чтение логического уровня вывода из регистра `IDR`;
- Параллельную групповую запись и чтение нескольких линий одного порта по битовой маске;
- Поддержку безопасной повторной инициализации выводов с изменением режима работы без возникновения паразитных импульсных выбросов (переключений в некорректное состояние) на выходах в процессе записи в управляющие регистры.

3. Аппаратная платформа и схема подключения стенда SDK-1.1M

Учебный стенд-конструктор SDK-1.1M (модификация SDK-1.1MC.427) оборудован процессорным модулем с 32-разрядным микроконтроллером `STM32F427VIT6` в корпусе LQFP100. Микроконтроллер построен на базе процессорного ядра ARM Cortex-M4 с аппаратным блоком вычислений с плавающей точкой (FPU) и способен работать на частоте до 180 МГц.

Для реализации поставленной задачи задействованы аппаратные компоненты боковой панели стенда SDK-1.1M, подключенные к микроконтроллеру в соответствии с электрической принципиальной схемой:

Таблица 1 — Аппаратные ресурсы стенда SDK-1.1M, используемые в работе

Элемент стенда	Контакт MCU	Режим GPIO	Активный уровень	Схемотехнические особенности
Кнопка боковой панели	PC15	Вход (Input) с программной подтяжкой Pull-Up	Низкий (логический 0 при нажатии)	Кнопка SB1 при нажатии замыкает контакт PC15 на шину земли GND. Схемотехнический фильтр не устраняет дребезг

Элемент стенда	Контакт MCU	Режим GPIO	Активный уровень	Схемотехнические особенности
				полностью, требуется программное подавление.
Зеленый светодиод	PD13	Выход Push-Pull, Low speed	Высокий (логическая 1 — свечение)	Индикатор VD6 «РАБОТА». Включается высоким логическим уровнем через токоограничивающий резистор 330 Ом.
Желтый светодиод	PD14	Выход Push-Pull, Low speed	Высокий (логическая 1 — свечение)	Секция YARC двухцветного индикатора VD7 «РЕЖИМ».
Красный светодиод	PD15	Выход Push-Pull, Low speed	Высокий (логическая 1 — свечение)	Секция YCRA двухцветного индикатора VD7 «РЕЖИМ».

Линия кнопки PC15 в исходном состоянии подтягивается внутренним резистором микроконтроллера к шине питания 3.3 В (Pull-Up). При нажатии на кнопку линия замыкается на землю GND, формируя низкий логический уровень напряжения.

Светодиоды подключены катодами к общей шине через токоограничивающие резисторы номиналом 330 Ом, а анодами — к выводам микроконтроллера PD13, PD14, PD15. Следовательно, подача логической единицы на соответствующий вывод открывает цепь и вызывает свечение светодиода. Поскольку кристаллы желтого и красного свечения конструктивно объединены в одном корпусе двухцветного светодиода VD7 («РЕЖИМ»), драйвер индикации программно реализует взаимное исключение сигналов управления: включение одного цвета автоматически сбрасывает управляющий сигнал противоположного цвета.

4. Архитектурное описание программы и алгоритмов

Встроенное программное обеспечение спроектировано по модульному принципу в соответствии с требованиями к архитектуре встроенных систем. Программа разделена на слабо связанные компоненты:

- **Главный модуль (main):** отвечает за базовую инициализацию микроконтроллера, запуск тактирования и организацию главного цикла диспетчеризации.
- **Драйвер кнопки (button):** считывает состояние линии PC15, выполняет программную фильтрацию дребезга контактов и формирует дискретные события нажатия.
- **Драйвер светодиодов (led):** инкапсулирует непосредственное управление выводами PD13, PD14, PD15 и обеспечивает выполнение световых эффектов (одиночные вспышки, периодическое мигание заданной частоты и длительности) в неблокирующем режиме.

- **Конечный автомат кодового замка (lock_fsm):** реализует всю прикладную логику задачи, обрабатывает поступающие события кнопки, контролирует таймауты и переключает режимы системы.
- **Регистровый драйвер GPIO (my_gpio):** аппаратно-зависимый уровень управления портами микроконтроллера через прямую запись и чтение системных регистров ARM Cortex-M4.

4.1. Диспетчеризация и принципы кооперативной многозадачности

Главный цикл программы организован без использования блокирующих функций ожидания типа `HAL_Delay()`. В начале каждой итерации цикла считывается текущее значение миллисекундного счетчика `SysTick` посредством вызова функции `HAL_GetTick()`. Далее управление последовательно передается каждому функциональному модулю:

1. Вызов функции обновления драйвера кнопки `button_update(now)` опрашивает состояние контакта, продвигает автомат подавления дребезга и возвращает событие (нет события, короткое нажатие или длинное нажатие).

2. Возвращенное событие передается в функцию обновления прикладного автомата `lock_fsm_update(evt, now)`, которая при необходимости меняет внутреннее состояние замка и формирует команды на включение светодиодных эффектов.

3. Вызов функции обновления светодиодов `led_update(now)` проверяет наступление запланированных моментов времени для выключения вспышек или переключения полупериодов мигания.

Время выполнения каждого шага составляет доли микросекунды, благодаря чему главный цикл успевает совершать десятки тысяч итераций в секунду. Проверка истечения временных интервалов производится по беззнаковой разности между текущим и сохраненным временем: `(current_tick - start_tick) >= DURATION`. Данный подход гарантирует корректность отсчета времени даже в момент аппаратного переполнения 32-разрядного счетчика `HAL_GetTick()`, происходящего раз в 49.7 суток.

```
/* Листинг 4.1. Диспетчерский цикл кооперативной многозадачности (main.c) */
while (1)
{
    uint32_t now = HAL_GetTick();

    /* 1. Опрос кнопки (дебаунсинг, классификация SHORT / LONG) */
    ButtonEvent evt = button_update(now);

    /* 2. Продвижение автомата кодового замка */
    lock_fsm_update(evt, now);

    /* 3. Диспетчеризация светодиодной индикации (мигание, вспышки) */
    led_update(now);
}
```

4.2. Алгоритм фильтрации дребезга и классификации нажатий кнопки

Механические контакты тактовой кнопки при замыкании и размыкании генерируют хаотическую серию переходных импульсов длительностью до нескольких десятков миллисекунд. Для надежного устранения дребезга и однозначного разделения коротких и длинных нажатий драйвер кнопки реализован в виде конечного автомата, включающего четыре состояния:

- **Состояние покоя (FSM_BTN_IDLE):** кнопка разомкнута, на входе PC15 присутствует высокий уровень напряжения 3.3 В (Pull-Up). При обнаружении перехода сигнала в

низкий логический уровень автомат фиксирует время начала контакта и переходит в состояние фильтрации замыкания.

- **Фильтрация замыкания (FSM_BTN_DEBOUNCE_DOWN):** автомат выжидает интервал подавления дребезга длительностью 35 мс. Если по истечении 35 мс на входе по-прежнему сохраняется низкий уровень, нажатие признается истинным, фиксируется метка времени удержания кнопки, и автомат переходит в состояние удержания. Если уровень вернулся в высокий, воздействие классифицируется как помеха, и автомат возвращается в исходное состояние.
- **Состояние удержания (FSM_BTN_DOWN):** кнопка стабильно нажата. Автомат ожидает размыкания контактов (появление высокого уровня на входе PC15). При фиксации высокого уровня запоминается время начала размыкания, и автомат переходит в состояние фильтрации отпускания.
- **Фильтрация отпускания (FSM_BTN_DEBOUNCE_UP):** автомат выжидает интервал 35 мс для исключения дребезга при размыкании. Если высокий уровень подтверждается, кнопка считается окончательно отпущенной. Вычисляется полная длительность нажатия (разность между временем отпускания и временем первоначального замыкания), после чего классифицируется событие: если длительность превышает 450 мс, генерируется событие длинного нажатия (BTN_EVENT_LONG); если длительность находится в диапазоне от 50 до 449 мс, генерируется событие короткого нажатия (BTN_EVENT_SHORT); импульсы короче 50 мс отсекаются как ложные. После генерации события автомат возвращается в состояние покоя.

```
/* Листинг 4.2. Подавление дребезга и классификация нажатий кнопки (button.c) */
ButtonEvent button_update(uint32_t current_tick)
{
    ButtonEvent event = BTN_EVENT_NONE;
    bool pressed = hw_read_button_pressed();

    switch (s_state) {
        case FSM_BTN_IDLE:
            if (pressed) {
                s_state = FSM_BTN_DEBOUNCE_DOWN;
                s_state_start_tick = current_tick;
            }
            break;

        case FSM_BTN_DEBOUNCE_DOWN:
            if ((current_tick - s_state_start_tick) >= DEBOUNCE_TIME_MS) {
                if (pressed) {
                    s_state = FSM_BTN_DOWN;
                    s_press_start_tick = current_tick;
                } else {
                    s_state = FSM_BTN_IDLE;
                }
            }
            break;

        case FSM_BTN_DOWN:
            if (!pressed) {
                s_state = FSM_BTN_DEBOUNCE_UP;
                s_state_start_tick = current_tick;
            }
            break;

        case FSM_BTN_DEBOUNCE_UP:
            if ((current_tick - s_state_start_tick) >= DEBOUNCE_TIME_MS) {
                if (!pressed) {
                    uint32_t duration = current_tick - s_press_start_tick;
                    if (duration >= LONG_PRESS_TIME_MS) {
                        event = BTN_EVENT_LONG;
```

```
                    } else if (duration >= MIN_PRESS_TIME_MS) {
                        event = BTN_EVENT_SHORT;
                    }
                    s_state = FSM_BTN_IDLE;
                } else {
                    s_state = FSM_BTN_DOWN;
                }
            }
            break;
    }
    return event;
}
```

4.3. Организация светодиодной индикации

Драйвер светодиодов обеспечивает раздельное управление статическими уровнями и динамическими временными эффектами без задержки основного потока вычислений:

- **Одиночная вспышка:** при вызове функции запуска вспышки целевой светодиод переводится в активное состояние, а в структуру эффекта сохраняется целевая временная метка окончания. Функция обновления проверяет достижение заданного времени и выключает светодиод.
- **Циклическое мигание:** при запуске мигания сохраняются период полупериода и общее время активности. Функция обновления производит инверсию логического состояния светодиода через каждый полупериод и полностью гасит светодиод по истечении общего времени тревоги.
- **Взаимная блокировка двухцветного индикатора:** при подаче команды на включение желтого канала (PD14) управляющий бит красного канала (PD15) принудительно сбрасывается в 0, и наоборот. Это исключает смешение цветов и перегрузку выходов микроконтроллера.

4.4. Логика конечного автомата кодового замка

Прикладной автомат кодового замка оперирует четырьмя логическими состояниями:

- **Состояние ожидания ввода (LOCK_STATE_INPUT):** замок готов к приему нажатий. Если пользователь уже ввел хотя бы один элемент, но с момента последнего нажатия прошло более 5 секунд, фиксируется таймаут неактивности: индекс сбрасывается в 0, запускается длинный световой импульс желтого светодиода длительностью 1500 мс, и автомат переходит в состояние отработки таймаута. При поступлении события нажатия оно сопоставляется с эталонным массивом кода. При совпадении индекс увеличивается; если введены все 8 элементов, включается зеленый светодиод на 3 секунды, а автомат переходит в состояние открытого замка; если последовательность не завершена, формируется короткая вспышка желтого светодиода на 200 мс. При ошибочном нажатии индекс сбрасывается в 0, инкрементируется счетчик ошибок: если число ошибок достигло 3, запускается 5.5-секундное мигание красного светодиода, и автомат переходит в состояние тревоги; если ошибок менее 3, выдается одиночная вспышка красного светодиода на 250 мс.
- **Состояние открытого замка (LOCK_STATE_UNLOCKED):** зеленый светодиод непрерывно горит на протяжении 3 секунд, все нажатия на кнопку игнорируются. По истечении интервала светодиод гаснет, и автомат возвращается в состояние ожидания ввода.
- **Состояние тревоги и блокировки (LOCK_STATE_ALARM_BLOCKED):** красный светодиод мигает с частотой 2.5 Гц в течение 5.5 секунд. Нажатия на кнопку не

обрабатываются. По окончании тревоги автомат возвращается в исходное состояние с обнуленным счетчиком ошибок.

- **Состояние отработки импульса таймаута (LOCK_STATE_TIMEOUT_PULSE):** желтый светодиод отрабатывает непрерывный импульс длительностью 1.5 с, сигнализируя о сбросе неполного ввода. По завершении импульса автомат возвращается в режим ожидания нового ввода.

```
/* Листинг 4.3. Логика переходов автомата кодового замка (lock_fsm.c) */
void lock_fsm_update(ButtonEvent btn_event, uint32_t current_tick)
{
    switch (s_lock_state) {
        case LOCK_STATE_INPUT:
            if (s_input_index > 0 && (current_tick - s_last_press_tick) >=
INPUT_TIMEOUT_MS) {
                s_input_index = 0;
                led_trigger_flash(LED_YELLOW, TIMEOUT_PULSE_MS);
                s_lock_state = LOCK_STATE_TIMEOUT_PULSE;
                s_state_end_tick = current_tick + TIMEOUT_PULSE_MS;
                return;
            }
            if (btn_event != BTN_EVENT_NONE) {
                s_last_press_tick = current_tick;
                if (btn_event == SECRET_CODE[s_input_index]) {
                    s_input_index++;
                    if (s_input_index >= CODE_LENGTH) {
                        s_input_index = 0;
                        s_error_count = 0;
                        led_stop_effects();
                        led_set(LED_GREEN, LED_STATE_ON);
                        s_lock_state = LOCK_STATE_UNLOCKED;
                        s_state_end_tick = current_tick + UNLOCKED_TIME_MS;
                    } else {
                        led_trigger_flash(LED_YELLOW, FLASH_YELLOW_MS);
                    }
                } else {
                    s_input_index = 0;
                    s_error_count++;
                    if (s_error_count >= 3) {
                        s_error_count = 0;
                        led_start_blink(LED_RED, 200, ALARM_TIME_MS);
                        s_lock_state = LOCK_STATE_ALARM_BLOCKED;
                        s_state_end_tick = current_tick + ALARM_TIME_MS;
                    } else {
                        led_trigger_flash(LED_RED, FLASH_RED_MS);
                    }
                }
            }
            break;
        case LOCK_STATE_UNLOCKED:
            if ((int32_t)(current_tick - s_state_end_tick) >= 0) {
                led_set(LED_GREEN, LED_STATE_OFF);
                s_lock_state = LOCK_STATE_INPUT;
            }
            break;
        case LOCK_STATE_ALARM_BLOCKED:
            if ((int32_t)(current_tick - s_state_end_tick) >= 0 && !led_is_blinking()) {
                s_lock_state = LOCK_STATE_INPUT;
            }
            break;
        case LOCK_STATE_TIMEOUT_PULSE:
            if ((int32_t)(current_tick - s_state_end_tick) >= 0) {
                s_lock_state = LOCK_STATE_INPUT;
            }
```

```
            break;
    }
}
```

5. Анализ регистров GPIO STM32F427 и реализация драйвера my_gpio

5.1. Архитектура и назначение регистров GPIO

В микроконтроллере `STM32F427VIT6` периферийные порты ввода-вывода подключены к высокоскоростной шине AHB1 с частотой тактирования до 180 МГц. Каждый порт управляется набором 32-разрядных регистров, отображенных на системное адресное пространство памяти:

Таблица 2 — Управляющие и информационные регистры блока GPIO STM32F4

Регистр	Смещение	Назначение	Разрядность поля на вывод	Функциональное описание
MODER	`0x00`	Режим работы вывода (Port mode register)	2 бита	Задает конфигурацию вывода: 00 — цифровой вход (Input); 01 — цифровой выход общего назначения (General purpose output); 10 — альтернативная функция периферии (Alternate function); 11 — аналоговый вход (Analog).
OTYPER	`0x04`	Схемотехника выходного каскада (Output type register)	1 бит	0 — двухтактный выход (Push-Pull, активная подтяжка к питанию и земле); 1 — выход с открытым стоком (Open-Drain, активное притягивание только к земле).
OSPEEDR	`0x08`	Скорость нарастания фронтов (Output speed register)	2 бита	Определяет крутизну фронтов выходного сигнала: 00 — Low speed (2 МГц); 01 — Medium speed (25 МГц); 10 — Fast speed (50 МГц); 11 — High speed (100 МГц).
PUPDR	`0x0C`	Конфигурация внутренних	2 бита	Подключение внутренних резисторов подтяжки

Регистр	Смещение	Назначение	Разрядность поля на вывод	Функциональное описание
		подтяжек (Pull-up/pull-down register)		номиналом около 40 кОм: 00 — подтяжка отключена (Floating); 01 — подтяжка к шине питания VDD (Pull-Up); 10 — подтяжка к шине земли GND (Pull-Down).
IDR	`0x10`	Входные данные порта (Input data register)	1 бит	Регистр только для чтения. Содержит логические уровни напряжений на внешних выводах порта после прохождения через встроенный триггер Шмитта.
ODR	`0x14`	Выходные данные порта (Output data register)	1 бит	Регистр чтения и записи. Содержит значения, записанные в выходные триггеры-защелки порта.
BSRR	`0x18`	Атомарная установка и сброс бит (Bit set/reset register)	2 бита	Регистр только для записи. Младшие 16 бит (BS0..BS15) устанавливают соответствующие выводы в 1; старшие 16 бит (BR0..BR15) сбрасывают выводы в 0.

5.2. Реализация безопасной инициализации без выбросов (Glitch-Free Reconfiguration)

При конфигурировании портов ввода-вывода микроконтроллера классическая последовательность настройки, применяемая во многих типовых библиотеках, сначала переводит вывод в режим выхода через регистр `MODER`, а затем записывает требуемый уровень в регистр данных. Это приводит к возникновению опасного схемотехнического дефекта — паразитного импульсного выброса (glitch). Если до момента настройки вывод находился в состоянии входа, значение его внутренней защелки `ODR` могло быть неопределенным или нулевым. В момент активации выходного буфера на контакте на доли микросекунды появляется нежелательный уровень напряжения до тех пор, пока программа не запишет правильное значение.

В разработанном драйвере `my_gpio` реализован принцип гарантированной бесконфликтной реконфигурации:

1. Перед модификацией режима работы вывода анализируется желаемое начальное состояние. Если вывод настраивается как выход, требуемый логический уровень (0 или 1) предварительно защелкивается через регистр BSRR еще при выключенном выходном каскаде.
2. Производится настройка вспомогательных параметров: схемотехники каскада (OTYPER), скорости переключения (OSPEEDR) и внутренних подтягивающих резисторов (PUPDR).
3. Только на завершающем этапе модифицируются биты регистра MODER, разрешая работу выходного буфера. В момент подключения каскада к физическому выводу на нем уже зафиксирован целевой уровень напряжения, что полностью исключает возникновение паразитных импульсов и дребезга при инициализации.

```
/* Листинг 5.1. Безопасная инициализация портов ввода-вывода (my_gpio.c) */
void my_gpio_init(GPIO_TypeDef *GPIOx, const MyGPIOConfig *config)
{
    if (GPIOx == NULL || config == NULL) return;

    for (uint32_t pos = 0; pos < 16; pos++) {
        uint16_t pin = (1U << pos);
        if (config->pin_mask & pin) {
            /* 1. Предварительная фиксация уровня в регистре BSRR до переключения режима */
            if (config->mode == MY_GPIO_MODE_OUTPUT_PP || config->mode ==
MY_GPIO_MODE_OUTPUT_OD) {
                GPIOx->BSRR = config->init_state ? pin : (pin << 16U);
            }
            /* 2. Конфигурация каскада: Push-Pull / Open-Drain (OTYPER) */
            if (config->mode == MY_GPIO_MODE_OUTPUT_OD) GPIOx->OTYPER |= (1U << pos);
            else GPIOx->OTYPER &= ~(1U << pos);

            /* 3. Скорость нарастания фронтов (OSPEEDR) */
            GPIOx->OSPEEDR &= ~(0x3U << (pos * 2U));
            GPIOx->OSPEEDR |= ((uint32_t)config->speed << (pos * 2U));

            /* 4. Конфигурация подтягивающих резисторов (PUPDR) */
            GPIOx->PUPDR &= ~(0x3U << (pos * 2U));
            GPIOx->PUPDR |= ((uint32_t)config->pull << (pos * 2U));

            /* 5. Переключение режима вывода (MODER) */
            uint32_t mode_val = (config->mode == MY_GPIO_MODE_OUTPUT_PP ||
                                 config->mode == MY_GPIO_MODE_OUTPUT_OD) ? 0x1U : 0x0U;
            GPIOx->MODER &= ~(0x3U << (pos * 2U));
            GPIOx->MODER |= (mode_val << (pos * 2U));
        }
    }
}
```

5.3. Преимущества использования регистра BSRR над ODR

Использование регистра ODR для раздельного управления отдельными битами порта требует выполнения составной операции чтения-модификации-записи (Read-Modify-Write). Для установки одного бита процессор обязан считать значение регистра ODR, выполнить побитовое логическое сложение с маской и записать результат обратно. Если между чтением и записью управление передается обработчику прерывания, который изменяет состояние другого вывода того же порта, то при возврате в прерванный поток новое значение будет перезаписано старыми считанными данными. Это создает классическое состояние гонки данных (Race Condition).

Регистр BSRR аппаратно решает эту проблему. Поскольку биты установки (0–15) и биты сброса (16–31) разнесены по раздельным полям, а запись нуля аппаратно игнорируется, изменение любого количества линий порта выполняется строго за один

машинный такт шины посредством одной ассемблерной команды записи. Это обеспечивает абсолютную аппаратную атомарность операций и исключает необходимость введения критических секций и запрета прерываний при манипуляциях с портами GPIO.

```
/* Листинг 5.2. Атомарное управление состоянием вывода через регистр BSRR (my_gpio.c) */
void my_gpio_write_pin(GPIO_TypeDef *GPIOx, uint16_t pin, uint8_t state)
{
    if (GPIOx == NULL) return;

    if (state) {
        GPIOx->BSRR = (uint32_t)pin;           /* Биты BS0..BS15: установка 1 за 1 такт */
    } else {
        GPIOx->BSRR = ((uint32_t)pin << 16U);   /* Биты BR0..BR15: сброс в 0 за 1 такт */
    }
}

uint8_t my_gpio_read_pin(GPIO_TypeDef *GPIOx, uint16_t pin)
{
    if (GPIOx == NULL) return 0U;
    return ((GPIOx->IDR & pin) != 0U) ? 1U : 0U;
}
```

6. Методика и результаты экспериментальной верификации

Проверка работоспособности встроенного программного обеспечения проводилась на лабораторном стенде SDK-1.1M (микроконтроллер `STM32F427VIT6`). Временные характеристики и логика работы проверялись во всех штатных и нештатных режимах функционирования системы.

Таблица 3 — Протокол тестирования программного комплекса «Кодовый замок»

№	Тестовый сценарий	Входное воздействие	Ожидаемая реакция системы	Фактический результат на стенде SDK-1.1M
1	Ввод правильных промежуточных элементов	Поочередный ввод: К (150 мс), затем К (200 мс), затем Д (600 мс) с паузами менее 5 секунд	Одиночная вспышка желтого светодиода длительностью 200 мс на каждое корректное нажатие	Желтый светодиод четко вспыхивает на 200 мс строго после каждого надежного отпускания кнопки
2	Полный корректный ввод (открытие замка)	Ввод всей цепочки из 8 нажатий: [К, К, Д, К, Д, Д, К, К]	Включение зеленого светодиода ровно на 3 секунды, игнорирование нажатий на кнопку, возврат в исходное состояние	Зеленый светодиод включился на 3 секунды; нажатия на кнопку во время его горения полностью игнорировались; замок вернулся в режим ожидания

№	Тестовый сценарий	Входное воздействие	Ожидаемая реакция системы	Фактический результат на стенде SDK-1.1M
3	Одиночная ошибка ввода	Ввод длинного нажатия Д вместо ожидаемого короткого К на первом или промежуточном шаге	Одиночная вспышка красного светодиода длительностью 250 мс, сброс позиции ввода в начало	Красный светодиод кратковременно вспыхнул на 250 мс, введенная последовательность сбросилась, счетчик ошибок равен 1
4	Трехкратная ошибка ввода (тревога)	Три подряд ошибочных ввода элементов кодовой последовательности	Включение режима тревоги: непрерывное мигание красного светодиода длительностью не менее 5 с (5.5 с), блокировка ввода	Красный светодиод мигал с полупериодом 200 мс в течение 5.5 секунд; любые нажатия на кнопку во время тревоги не воспринимались; после тревоги счетчик ошибок обнулился
5	Таймаут неполного ввода	Ввод первых двух правильных элементов, после чего пауза бездействия более 5 секунд	Длинный световой импульс желтого светодиода длительностью 1500 мс, сброс ввода в начало	Ровно через 5 секунд после последнего нажатия включился желтый светодиод на 1.5 секунды; после его выключения ввод сбросился в исходное состояние
6	Антидребезг контактов кнопки	Быстрые многократные нажатия кнопки, контактный дребезг на фронтах замыкания и размыкания	Полное отсечение дребезга, регистрация строго одного дискретного события на одно физическое нажатие	Ложные повторные срабатывания отсутствуют, дребезг контактов полностью подавлен программным фильтром с порогом 35 мс
7	Собственный регистровый драйвер `my_gpio`	Работа всей программы с активным макросом	Полная функциональная эквивалентность работы	Программа функционирует стабильно; управление линиями PC15, PD13,

№	Тестовый сценарий	Входное воздействие	Ожидаемая реакция системы	Фактический результат на стенде SDK-1.1M
		переключения USE_CUSTOM_GPIO 1	программы без вызовов стандартных функций HAL GPIO	PD14, PD15 осуществляется корректно; переходные импульсные выбросы отсутствуют

7. Выводы

В процессе выполнения лабораторной работы были всесторонне изучены принципы организации, схемотехники и программирования интерфейсов ввода-вывода общего назначения (GPIO) на примере стенда SDK-1.1M и микроконтроллера STM32F427VIT6:

1. Успешно спроектировано и реализовано законченное встроенное программное обеспечение для прикладной задачи «Кодовый замок» (Вариант 5) со строгим соблюдением всех временных и логических требований: ввод 8-элементной кодовой комбинации, индикация правильных элементов короткой желтой вспышкой (200 мс), открытие замка зеленым светодиодом на 3 секунды, индикация ошибок красной вспышкой (250 мс), 5.5-секундный режим тревоги при трех ошибках подряд и 1.5-секундный импульс желтого светодиода при таймауте бездействия 5 секунд.
2. Реализована неблокирующая архитектура кооперативной многозадачности, позволившая совместить в едином главном цикле опрос механической кнопки и обслуживание динамических светодиодных эффектов на базе системного таймера SysTick (HAL_GetTick()) без использования блокирующих задержек HAL_Delay() и без задействования прерываний.
3. Разработан устойчивый конечный автомат фильтрации дребезга контактов кнопки с временным порогом 35 мс и классификатором коротких (50–449 мс) и длинных (≥ 450 мс) нажатий.
4. Разработан независимый регистровый драйвер my_gpio, реализующий прямую работу с регистрами STM32 (MODER, OTYPER, OSPEEDR, PUPDR, BSRR, IDR), обладающий высоким быстродействием, аппаратной атомарностью операций через регистр BSRR и защитой от переходных импульсных выбросов напряжения при реконфигурации выводов (glitch-free).
5. Экспериментальная верификация на аппаратном стенде SDK-1.1M подтвердила стабильность функционирования и полное соответствие разработанной системы техническому заданию.